SMRFORGE

Info sheet · Rev 2026-10-14

DOWNLOAD PDF ↓

REACTOR-PHYSICS EVIDENCE

A number a reviewer can re-run.

Your physicists own the physics. SMRFORGE seals each result they produce as an evidence bundle anyone can **verify offline** and **re-derive from scratch** on the free, open engine. We don't compute your reactor; we make its numbers checkable. You don't have to trust us.

EVIDENCE BUNDLE

1.029633 k_eff

fidelity: screening · code-verification · advisory

case	IAEA-2D · full core
published reference	k = 1.029585
C–E (this mesh)	+4.8 pcm
converged (Richardson)	−3.0 pcm
signature	ed25519 ✓

$ smrforge verify

INTEGRITY OK ✓

1 · your code

Produces the number

Your physicists run the calculation in whatever code they use. SMRFORGE does not compute your reactor, it makes the result checkable.

2 · SMRFORGE

Seals it, then signs it

Inputs, result, and provenance hashed into one manifest and signed. The bundle is self-contained, so it needs no network and no vendor to check.

3 · any reviewer

Checks it cold

`$ smrforge verify` confirms integrity and origin offline, importing none of our physics. `$ smrforge reproduce` re-derives the number on the open engine, held to the sealed band. Forge a bundle that passes and get credited.

ENGAGEMENTS, WHAT WE TAKE ON

Bring us a number a reviewer won't take on faith.

Whatever the code it came from, we make the result checkable: sealed, verifiable offline, and re-derivable from scratch by whoever reviews it.

Independent confirmatory analysis FLAGSHIP

Send your criticality or shutdown-margin number. We re-derive it on the open engine and OpenMC and hand back a signed, reproducible confirmation your reviewer can re-run.

Evidence packaging

A number from your own code or tools, shipped as a sealed, verifiable, reproducible bundle for a submission.

Pre-submission V&V

Code verification, a V&V matrix, and the benchmark closures a regulator reads, stood up and re-runnable.

Pipeline integration

MCP and REST wiring into your, or a lab's, pipeline, so sealing a result is one call in a workflow you already run.

BRING US A NUMBER →

Full detail: how an engagement works →

VERIFICATION	IAEA-2D full core closes the published reference to ≈ **−3 pcm** (code-verification, not measured validation). Screening-grade, honestly labeled on every result.
LINEAGE	A result can cite the **sealed bundle it was built from**, so a reviewer walks the whole derivation with one command (`smrforge verify --chain`), custody end to end, not just the final number.
ENGINE	Screen fast in pure Python; escalate the **same inputs** to OpenMC, a lab-grade eigenvalue, free.
LICENSING	**Community** MIT (the whole engine + evidence spine, free) · **Pro** FSL-1.1-MIT (the signed assurance layer, coverage, watch, report, assess + an evidence console) · air-gap-safe.
RUN IT YOURSELF	Source-only install, by design: `git clone https://gitlab.com/smrforge/smrforge.git`, then `pip install -e .` (Python 3.11+, standard library, no dependencies; not on PyPI, on purpose). Run a capability from the command line (`smrforge run`), from Python (`run_capability`), or through the local REST/MCP server (`smrforge serve`). The in-tree engine is screening-grade and runs anywhere Python does; the lab-grade OpenMC path needs OpenMC and a continuous-energy library on your host. **Getting started:** gitlab.com/smrforge/smrforge/-/blob/main/docs/GETTING_STARTED.md

SMRFORGE · reactor physics you can prove

gitlab.com/smrforge · support@smrforge.io · smrforge.io